📁 Arduino – Sistema Purificatore OffGas

Documento operativo e descrittivo

Questo documento descrive l'implementazione del firmware Arduino utilizzato nel sistema **OffGas**.

Il firmware ha il compito di gestire:

1️⃣ lettura del sensore di gas **MQ-2**
2️⃣ comunicazione Bluetooth con il **Bridge Python**
3️⃣ attivazione della **ventola di ventilazione**
4️⃣ gestione dello **stato del dispositivo**
5️⃣ visualizzazione delle informazioni tramite **display OLED**

Arduino rappresenta il **livello hardware (edge device)** del sistema IoT OffGas.

Panoramica del ruolo di Arduino

Nel sistema OffGas Arduino è collegato a:

- sensore **MQ-2**
- modulo Bluetooth **HC-05**
- **ventola DC**
- **LED di stato**
- **display OLED I2C**

Arduino **non prende decisioni autonome** sulla qualità dell'aria.

La logica di controllo è distribuita:

Componente	Ruolo
Arduino	acquisizione sensore e attuazione ventola
Bridge Python	gateway hardware
Node-RED	logica decisionale

Arduino si limita quindi a:

- inviare il valore del gas
- ricevere i comandi di accensione o spegnimento della ventola
- visualizzare lo stato del sistema sul display OLED

Architettura hardware

I componenti principali collegati ad Arduino sono:

Componente	Funzione
MQ-2	sensore gas
HC-05	comunicazione Bluetooth
Ventola DC	ventilazione
LED	indicatore stato ventola
Display OLED	visualizzazione dati

Definizione dei PIN

Nel firmware i pin utilizzati sono definiti all'inizio del programma.

```
const int mq2Pin = A1;
const int bluetoothRxPin = 10;
const int bluetoothTxPin = 11;
const int PIN_VENTOLA = 9;
```

Pin	Funzione
A1	ingresso analogico sensore MQ-2
10	RX Bluetooth
11	TX Bluetooth
9	controllo ventola

Display OLED

Il sistema include un **display OLED I2C** utilizzato per visualizzare:

- valore del gas rilevato
- stato della ventola

Il display utilizza la libreria:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>
```

Configurazione:

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SH1106G display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
```

L'indirizzo I2C utilizzato è:

0x3C

Il display mostra:

OffGas
Gas: <valore>
Fan: ON/OFF

Comunicazione Bluetooth

La comunicazione con il Bridge avviene tramite **SoftwareSerial**.

```
#include <SoftwareSerial.h>
```

Inizializzazione:

```
SoftwareSerial bluetooth(bluetoothRxPin, bluetoothTxPin);
```

Velocità di comunicazione:

```
bluetooth.begin(9600);
```

Questa velocità deve corrispondere a quella utilizzata dal Bridge Python:

```
BAUD_RATE = 9600
```

Parametri operativi

Il firmware utilizza una lettura periodica del sensore.

```
const unsigned long intervalloLettura = 5000;
```

Significato:

5000 ms = 5 secondi

Arduino invia quindi un nuovo valore del gas ogni **5 secondi**.

Variabili di stato

Il firmware mantiene alcune variabili per tracciare lo stato del sistema.

```
unsigned long ultimoTempoLettura = 0;
bool fanState = false;
int lastGasValue = 0;
bool displayDirty = true;
```

Variabile	Funzione
ultimoTempoLettura	ultimo momento di lettura del sensore
fanState	stato corrente della ventola
lastGasValue	ultimo valore di gas rilevato
displayDirty	indica quando il display deve essere aggiornato

Inizializzazione del sistema (setup)

La funzione `setup()` inizializza i componenti hardware.

```
pinMode(mq2Pin, INPUT);
pinMode(PIN_VENTOLA, OUTPUT);
pinMode(LED_BUILTIN, OUTPUT);
```

Inizializzazione delle comunicazioni seriali:

```
Serial.begin(9600);
bluetooth.begin(9600);
```

Inizializzazione del display:

```
display.begin(0x3C, true);
```

Pulizia dello schermo:

```
display.clearDisplay();
display.setTextColor(SH110X_WHITE);
```

Ciclo principale del firmware

Arduino utilizza il ciclo infinito:

```
void loop()
```

All'interno del ciclo vengono gestite tre funzioni principali:

1. invio del valore del sensore
2. ricezione dei comandi dal Bridge
3. aggiornamento del display OLED

1️⃣ Invio periodico del valore MQ-2

La temporizzazione è gestita tramite:

millis()

Controllo intervallo:

if (now - ultimoTempoLettura >= intervalloLettura)

Lettura sensore:

int valore = analogRead(mq2Pin);

Range valori:

0 – 1023

Invio del valore al Bridge

Formato del messaggio:

MQ2:<valore>

Codice:

```
bluetooth.print("MQ2:");
bluetooth.println(valore);
```

Esempio:

MQ2:178

Il Bridge effettua il parsing tramite:

if read_val.startswith("MQ2:")

Debug sulla seriale

Il valore del sensore viene stampato anche sulla seriale USB.

```
Serial.print("MQ2:");
Serial.println(valore);
```

Questo permette il monitoraggio tramite **Serial Monitor**.

---

2 Ricezione comandi dal Bridge

Controllo presenza dati:

```
if (bluetooth.available())
```

Lettura comando:

```
String comando = bluetooth.readStringUntil('\n');
```

Il Bridge invia i comandi con newline:

```
FAN_ON
FAN_OFF
```

Comando accensione ventola

```
if (comando.equalsIgnoreCase("FAN_ON"))
```

Azioni eseguite:

```
digitalWrite(PIN_VENTOLA, HIGH);
digitalWrite(LED_BUILTIN, HIGH);
fanState = true;
```

Comando spegnimento ventola

else if (comando.equalsIgnoreCase("FAN_OFF"))

Azioni eseguite:

```
digitalWrite(PIN_VENTOLA, LOW);
digitalWrite(LED_BUILTIN, LOW);
fanState = false;
```

Ti spiego meglio il motivo tecnico e la logica di funzionamento, così la spiegazione è più chiara anche per una relazione universitaria.

3️⃣ Aggiornamento del display OLED

Il display OLED non viene aggiornato continuamente ad ogni ciclo del programma.

Questo è necessario per evitare conflitti tra due componenti del sistema:

- la comunicazione Bluetooth tramite SoftwareSerial
- la comunicazione I2C utilizzata dal display OLED

Entrambi questi meccanismi utilizzano interrupt e timing precisi per funzionare correttamente.

Problema tecnico

Il display OLED utilizza il bus I2C.

Quando Arduino aggiorna lo schermo tramite la funzione:

display.display()

viene trasferita una grande quantità di dati (circa 1 KB di buffer grafico) attraverso il bus I2C.

Durante questo trasferimento Arduino rimane occupato per alcuni millisecondi.

Nel frattempo il modulo Bluetooth HC-05 comunica tramite SoftwareSerial, che richiede un timing molto preciso per ricevere i byte.

Se Arduino è impegnato ad aggiornare il display mentre arrivano dati Bluetooth, possono verificarsi problemi come:

- perdita di byte
- comandi incompleti
- ritardi nell'attivazione della ventola

Per evitare questi problemi, il firmware aggiorna il display solo quando è sicuro farlo.

Variabile di controllo:

displayDirty

Per gestire l'aggiornamento del display viene utilizzata una variabile di stato:

displayDirty

Questa variabile indica se il contenuto dello schermo deve essere aggiornato.

Il valore diventa true quando cambia qualcosa nel sistema, ad esempio:

- viene letto un nuovo valore di gas
- cambia lo stato della ventola

Esempio nel codice:

displayDirty = true;

In questo modo Arduino sa che il display deve essere aggiornato, ma non lo fa immediatamente.

Condizione di aggiornamento del display

L'aggiornamento avviene solo quando sono soddisfatte due condizioni:

if (displayDirty && !bluetooth.available())

Significato delle due condizioni:

1️⃣

displayDirty

Il display viene aggiornato solo se il contenuto è cambiato.

Questo evita aggiornamenti inutili e riduce il carico sul microcontrollore.

2️⃣

!bluetooth.available()

Arduino verifica che non ci siano dati Bluetooth in arrivo.

Se il modulo HC-05 sta inviando dati, l'aggiornamento del display viene rimandato.

Questo evita che la funzione display.display() interferisca con la ricezione dei comandi.

Funzionamento complessivo

Il comportamento del firmware è quindi il seguente:

1️⃣ Arduino legge il sensore MQ-2

2️⃣ Il valore viene inviato al Bridge tramite Bluetooth

3️⃣ Se il valore cambia, displayDirty viene impostato a true

4️⃣ Arduino controlla se il Bluetooth è libero

5️⃣ Se non stanno arrivando dati, il display viene aggiornato

6️⃣ displayDirty torna a false

Vantaggi di questo approccio

Questo metodo offre diversi vantaggi:

Maggiore stabilità della comunicazione Bluetooth

Si evita la perdita di dati durante la ricezione dei comandi.

Riduzione del carico sul microcontrollore

Il display viene aggiornato solo quando necessario.

Migliore reattività del sistema

I comandi di accensione o spegnimento della ventola vengono gestiti senza ritardi.

In sintesi

Il display OLED viene aggiornato in modo controllato per evitare interferenze tra:

- Bluetooth SoftwareSerial
- comunicazione I2C del display

La variabile displayDirty permette di aggiornare lo schermo solo quando necessario e quando il canale Bluetooth è libero, garantendo il corretto funzionamento dell'intero sistema OffGas.

Funzione aggiornaDisplay()

void aggiornaDisplay()

La funzione aggiorna lo schermo con:

- nome sistema
- valore gas
- stato ventola

Esempio output:

OffGas
Gas: 178
Fan: ON

Comunicazione con il Bridge

La comunicazione segue questo schema.

Arduino → Bridge

Invio valore sensore:

MQ2:178

Bridge → Arduino

Comandi supportati:

FAN_ON
FAN_OFF

Il Bridge invia questi comandi quando riceve decisioni da **Node-RED**.

Flusso completo del sistema

Il funzionamento del sistema OffGas è il seguente.

1. Arduino legge il sensore MQ-2
2. Arduino invia il valore al Bridge

3️⃣ Bridge pubblica telemetria MQTT
4️⃣ Node-RED analizza i dati
5️⃣ Node-RED decide lo stato della ventola
6️⃣ Bridge invia comando ad Arduino
7️⃣ Arduino attiva o spegne la ventola

Vantaggi dell'architettura

Separare le responsabilità tra Arduino, Bridge e Node-RED offre diversi vantaggi.

Firmware semplice

Arduino esegue solo operazioni hardware.

Logica centralizzata

La logica di controllo è gestita da Node-RED.

Scalabilità

Il sistema può essere esteso facilmente a più garage.

Monitoraggio locale

Il display OLED consente di visualizzare localmente:

- valore del gas
- stato della ventola

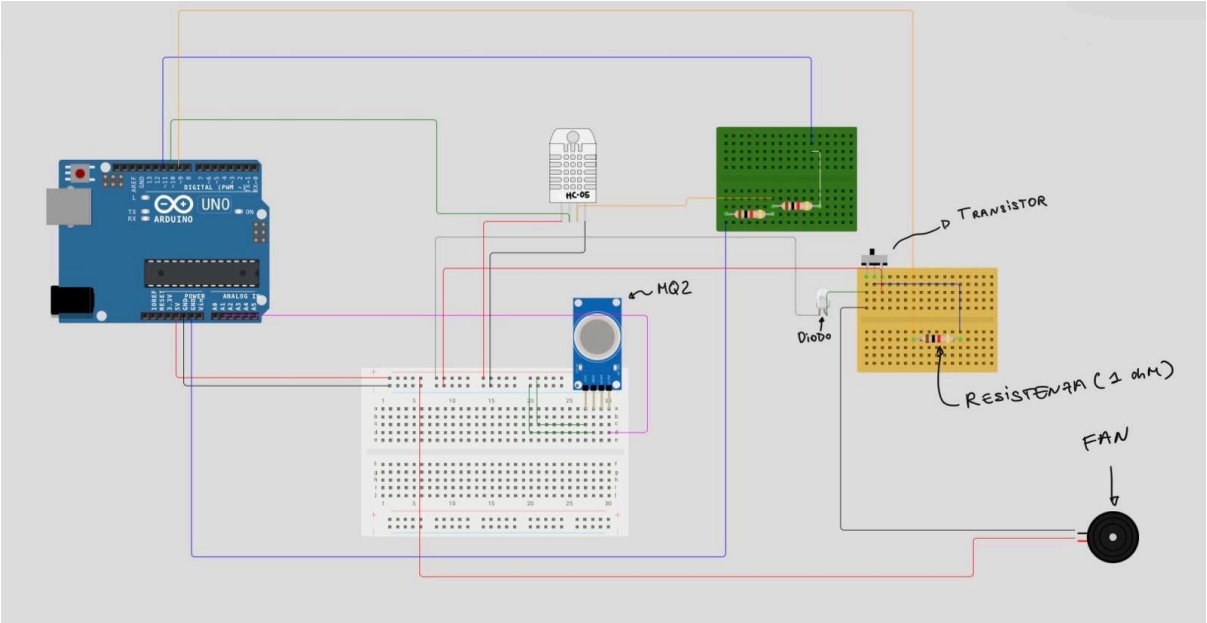
Checklist funzionale

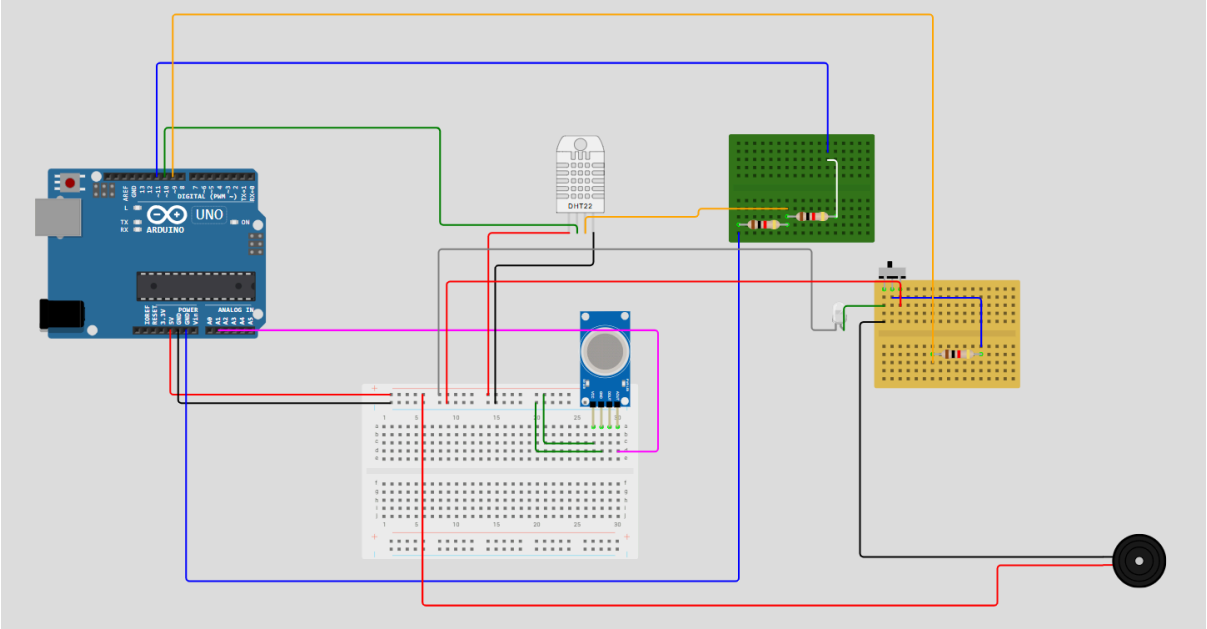
Il firmware Arduino implementa le seguenti funzionalità:

✔ Lettura del sensore MQ-2
✔ Invio valore gas via Bluetooth
✔ Ricezione comandi dal Bridge
✔ Controllo della ventola
✔ Indicazione visiva tramite LED
✔ Visualizzazione su display OLED
✔ Debug tramite seriale USB

CIRCUITO

(https://wokwi.com/projects/458136524962164737)





Descrizione del circuito hardware – Sistema OffGas

Panoramica

Il prototipo hardware del sistema OffGas è basato su **Arduino Uno** ed è progettato per monitorare la concentrazione di gas in un garage e attivare un sistema di ventilazione quando necessario.

Il sistema è composto da:

- un sensore di gas **MQ-2**
- un **modulo Bluetooth HC-05** per la comunicazione con il Bridge Python
- una **ventola DC** per la ventilazione
- un **transistor NPN** per il controllo della ventola
- un **diodo flyback** per la protezione del circuito
- resistenze di pull-up e di limitazione della corrente
- breadboard e cablaggio

Arduino funge da **nodo edge IoT**, acquisendo i dati dei sensori e ricevendo comandi dal Bridge per controllare la ventola.

Componenti utilizzate

Componente	Funzione
Arduino Uno	microcontrollore principale
Sensore MQ-2	rilevamento gas combustibili
Modulo Bluetooth HC-05	comunicazione con il Bridge Python
Ventola DC	ventilazione del garage
Transistor NPN	pilotaggio della ventola
Diodo flyback	protezione contro tensioni inverse
Resistenze	pull-up e limitazione corrente

Breadboard	prototipazione

Collegamenti dei sensori

Sensore gas MQ-2

Il sensore MQ-2 è utilizzato per misurare la concentrazione di gas nell'aria.

Collegamenti:

Pin MQ-2	Collegamento
VCC	5V Arduino
GND	GND Arduino
AO	A1 Arduino

Il pin **AO** fornisce un valore analogico proporzionale alla concentrazione di gas.

Comunicazione Bluetooth

La comunicazione con il sistema software è realizzata tramite il **modulo HC-05**.

Il modulo utilizza una comunicazione **seriale UART** con Arduino.

Collegamenti principali:

HC-05	Arduino
VCC	5V
GND	GND
TX	pin seriale Arduino
RX	pin seriale Arduino

Il modulo consente al Bridge Python di:

- ricevere i valori del sensore MQ-2

- inviare comandi di controllo della ventola

Circuito di controllo della ventola

La ventola non può essere collegata direttamente ad Arduino perché richiede una corrente superiore a quella fornibile dai pin digitali.

Per questo viene utilizzato un **transistor NPN come interruttore elettronico**.

Collegamenti del transistor

Pin transistor	Collegamento
Base	Arduino tramite resistenza
Collector	terminale negativo della ventola
Emitter	GND

La **resistenza sulla base** limita la corrente proveniente dal pin digitale di Arduino.

Collegamento della ventola

Ventola	Collegamento
positivo	alimentazione 5V
negativo	collector del transistor

Quando Arduino attiva il pin di controllo, il transistor conduce e la ventola si accende.

Diodo flyback

Il circuito include un **diodo flyback** collegato in parallelo alla ventola.

Funzione:

- proteggere il transistor
- proteggere Arduino
- assorbire la tensione inversa generata dal motore quando si spegne

Orientamento:

- catodo verso il lato positivo
- anodo verso il lato del transistor

Alimentazione

L'alimentazione del circuito è fornita dall'Arduino attraverso il pin **5V**.

La breadboard distribuisce:

- linea **5V**
- linea **GND**

a tutti i componenti del circuito.

Funzionamento del sistema

Il funzionamento del prototipo è il seguente:

1. Il sensore **MQ-2** misura la concentrazione di gas.
2. Arduino legge il valore analogico dal pin **A1**.
3. Il valore viene inviato via **Bluetooth (HC-05)** al Bridge Python.
4. Il Bridge pubblica i dati su **MQTT** verso Node-RED.
5. Node-RED analizza i dati e decide se attivare la ventilazione.
6. Se necessario, il Bridge invia il comando **FAN_ON** ad Arduino.
7. Arduino attiva il transistor che alimenta la **ventola DC**.

In caso contrario viene inviato il comando **FAN_OFF**, spegnendo il sistema di ventilazione.

Ruolo del prototipo nel sistema OffGas

Il prototipo rappresenta il **garage locale (G1)** nel sistema OffGas.

Il dispositivo:

- misura la qualità dell'aria
- invia i dati alla piattaforma IoT
- riceve comandi di controllo
- attiva il sistema di ventilazione quando necessario

In questo modo il garage diventa un **nodo IoT intelligente** integrato nella rete di monitoraggio distribuita.